# Übungsblatt 3

## Web-Anwendungen und Web-Architekturen

5430UE Web and Data Engineering

29. April 2026

Prof. Dr. Michael Granitzer, Tobias Fuchs

# Lernziele

- Unterscheiden von Kategorien von Web-Anwendungen und Zuordnung typischer Systeme
- Zentrale Prinzipien des Web Engineerings (z. B. SoC, SRP, DRY) verstehen, erkennen und anwenden
- Analyse grundlegender Architektur- und Betriebsaspekte von Web-Anwendungen

# Kategorien von Web-Anwendungen

# Übung 3.1: Kategorien von Web-Anwendungen

Ordnen Sie folgende Systeme einer Web-Anwendungskategorie zu (statisch, dynamisch server-seitig, Single-Page Application, Web-Service/API). Begründen Sie eine der Zuordnungen kurz.

| System | Kategorie |
|---|---|
| Universitäts-Raumbelegungsplan als HTML-Seite, wöchentlich neu generiert | ? |
| Online-Banking-Portal mit Echtzeit-Kontoübersicht | ? |
| Wetter-REST-API, die JSON zurückgibt | ? |
| Produktkatalog mit Suchfilter ohne Neuladen der Seite | ? |

# Übung 3.1: Kategorien von Web-Anwendungen — Lösung (1/2)

**Zuordnungen:**

| System | Kategorie |
|---|---|
| Universitäts-Raumbelegungsplan als HTML-Seite, wöchentlich neu generiert | Statisch (vorgerendert, unverändert vom Server ausgeliefert, keine serverseitige Logik, die Inhalte dynamisch erzeugt) |
| Online-Banking-Portal mit Echtzeit-Kontoübersicht | Dynamisch server-seitig (Session, Authentifizierung, Datenbankabfragen per Request) |
| Wetter-REST-API, die JSON zurückgibt | Web-Service/API (kein HTML, maschinenlesbare Daten) |
| Produktkatalog mit Suchfilter ohne Neuladen der Seite | Single-Page Application (JS übernimmt Routing und Datenabfragen im Browser) |

# Übung 3.1: Kategorien von Web-Anwendungen — Lösung (2/2)

**Begründungen:**

- **Raumbelegungsplan:** Eine statische Webanwendung liefert feste Inhalte (HTML/CSS), die unverändert vom Server an den Client übertragen werden. Die Inhalte werden nicht dynamisch erzeugt.
- **Online-Banking:** Inhalte werden auf dem Server zur Laufzeit erzeugt (z. B. durch Datenbankabfragen) und als fertige Seite an den Client gesendet.
- **Wetter-API:** Eine API liefert typischerweise strukturierte Daten (z.B. JSON oder XML), die primär für die Weiterverarbeitung durch Programme gedacht sind; die Darstellung übernimmt in der Regel der konsumierende Client.
- **Produktkatalog mit Filter:** Die Anwendung lädt initial eine Seite und aktualisiert Inhalte dynamisch im Browser, ohne die Seite neu zu laden.

# Web Engineering — Ziele und Prinzipien

# Übung 3.2: Web Engineering — Ziele und Prinzipien

Web Engineering ist die systematische Disziplin zur Entwicklung von Web-Anwendungen.

1. Geben Sie die Grundprinzipien des Web Engineering in eigenen Worten an.
2. Formulieren Sie in eigenen Worten: Was unterscheidet Web Engineering von klassischer Software-Entwicklung?
3. Nennen Sie drei typische Qualitätsziele, die Web-Anwendungen im Gegensatz zu Desktop-Software besonders stark betonen.
4. Ein Unternehmen entwickelt seine Web-Anwendung ohne Versionskontrolle, ohne Tests und ohne dokumentierte Anforderungen. Welche konkreten Probleme entstehen kurz- und langfristig?

# Übung 3.2: Web Engineering — Lösung (1/3)

1. **Grundprinzipien:**

   - Klar definierte Ziele und Anforderungen
   - Systematische Entwicklung in Phasen
   - Kontinuierliche Überwachung des gesamten Entwicklungsprozesses
   - Planung der Wartung (Post-Launch-Plan)

2. **Web Engineering vs. klassische Softwareentwicklung:**

   Web Engineering muss mit mehr Unsicherheiten und Besonderheiten umgehen als klassische Softwareentwicklung. Web Engineering behandelt die Besonderheiten von Web-Anwendungen: heterogene Clients (Browser, Mobilgeräte), zustandsloses HTTP-Protokoll, öffentliche Erreichbarkeit, schnelle Release-Zyklen, verteilte Teams und hohe Nutzerzahlen.

   Klassische SE optimiert oft für einen definierten Client-Typ und ein kontrolliertes Deployment-Umfeld (fixe Systemanforderungen vs. Browser, Geräte, Bildschirmgrößen,...).

# Übung 3.2: Web Engineering — Lösung (2/3)

3. **Typische Web-Qualitätsziele:**

- **Skalierbarkeit:** Tausende bis Millionen gleichzeitiger Nutzer
- **Accessibility/Usability:** Unterschiedliche Endgeräte, Screenreader, Barrierefreiheit
- **Performance/Ladezeit:** Nutzer verlassen Seiten nach mehr als 3 Sekunden Ladezeit

4. **Probleme ohne systematisches Vorgehen:**

# Übung 3.2: Web Engineering — Lösung (3/3)

- **Langfristige Probleme:**
  - **Akkumulation technischer Schulden:**
    Unsicherer, schlecht strukturierter Code wächst an und erschwert langfristig Wartung und Weiterentwicklung.
  - **Hohe Änderungs- und Wartungskosten:**
    Jede neue Funktion wird riskanter und aufwendiger, da Auswirkungen von Änderungen schwer abschätzbar sind.
  - **Dauerhafte Regressionen und sinkende Qualität:**
    Fehler werden immer wieder neu eingebaut und oft erst spät entdeckt, häufig durch Endnutzer.
  - **Abhängigkeit von Einzelpersonen ("Wissensinseln"):**
    Wissen existiert nur in den Köpfen einzelner Entwickler, was bei Ausfall oder Weggang kritisch wird.
  - **Erschwertes Onboarding neuer Entwickler:**
    Neue Teammitglieder benötigen viel Zeit, um sich ohne Dokumentation und klare Struktur einzuarbeiten.
  - **Fehlende Auditierbarkeit und Compliance-Probleme:**
    Änderungen und Verantwortlichkeiten sind nicht nachvollziehbar, was bei rechtlichen oder regulatorischen Anforderungen problematisch ist.
  - **Geringe Skalierbarkeit des Entwicklungsprozesses:**
    Mit wachsendem Team und Projektumfang steigt der Koordinationsaufwand stark an und führt zu Chaos.

# Ressourcen-Engpässe identifizieren

# Übung 3.3: Ressourcen-Engpässe identifizieren

Ein Online-Shop beobachtet die in der Tabelle aufgeführten Symptome. Geben Sie jeweils an, bei welcher Ressource - Rechenleistung (CPU), Arbeitsspeicher (RAM), persistenter Speicher (Storage), Netzwerk (I/O) - vermutlich der Engpass vorliegt, der zum jeweiligen Symptom führt. Begründen Sie jede Zuordnung in einem Satz.

| Symptom | Vermuteter Engpass |
|---|---|
| Antwortzeiten steigen bei komplexen Preisberechnungen | ? |
| Server stürzt ab wenn 50.000 Sessions gleichzeitig aktiv sind | ? |
| Produktbilder laden sehr langsam | ? |
| API-Requests von mobilen Clients laufen regelmäßig in Timeouts | ? |

# Übung 3.3: Ressourcen-Engpässe — Lösung (1/2)

**Zuordnungen:**

| Symptom | Vermuteter Engpass |
|---|---|
| Antwortzeiten steigen bei komplexen Preisberechnungen | CPU |
| Server stürzt ab wenn 50.000 Sessions gleichzeitig aktiv sind | RAM |
| Produktbilder laden sehr langsam | Storage I/O oder Netzwerk |
| API-Requests von mobilen Clients laufen regelmäßig in Timeouts | Netzwerk |

# Übung 3.3: Ressourcen-Engpässe — Lösung (2/2)

**Begründungen:**

- Komplexe Preisberechnungen erfordern viele Rechenoperationen und erhöhen dadurch die Auslastung der CPU.
- Viele gleichzeitige Sessions benötigen viel Arbeitsspeicher, sodass bei zu wenig RAM der Server instabil wird oder abstürzt (bei ca. 1 MB/Session = 50 GB RAM-Bedarf).
- Produktbilder sind große Dateien, deren langsames Laden entweder durch langsamen Datenträgerzugriff oder begrenzte Netzwerkbandbreite verursacht wird. (Oder fehlender CDN-Cache).
- Hohe Latenz oder geringe Bandbreite in der Netzwerkverbindung zwischen Client und Server.

# Vertikale vs. Horizontale Skalierung

# Übung 3.4: Vertikale vs. Horizontale Skalierung

Wir betrachten folgendes Szenario: *Ein Start-up betreibt seinen Web-Shop auf einem einzelnen Server (8 Kerne, 32 GB RAM). Mit großer werdender Popularität des Start-ups wächst der Traffic um Faktor 10.*

1. Beschreiben Sie eine **vertikale** Skalierungsmaßnahme und nennen Sie ihre Grenze.
2. Beschreiben Sie eine **horizontale** Skalierungsmaßnahme und die dabei entstehenden neuen Herausforderungen.
3. Warum ist Session-State ein kritisches Problem bei horizontaler Skalierung, und wie löst man es?

# Übung 3.4: Skalierung — Lösung

1. **Vertikale Skalierung:** Server auf 32 Kerne / 128 GB RAM aufrüsten.
    - **Grenze:** Physische Obergrenze (kein unendlich großer Einzelrechner) und exponentielle Kostensteigerung. Außerdem: Single Point of Failure bleibt bestehen.
2. **Horizontale Skalierung:** 5 identische Server hinter einem Load Balancer betreiben (verteilt eingehende Requests auf mehrere Instanzen).
    - **Neue Herausforderungen:** Session-State muss synchronisiert werden, Datenbankzugriffe müssen koordiniert werden (konsistenter Datenbankstand), Deployment wird komplexer.
3. **Session-State-Problem:** Wenn Nutzer A seinen Warenkorb auf Server 1 speichert, aber der nächste Request auf Server 2 landet, ist der Warenkorb weg.
    - **Lösung:** Session-Daten in einen zentralen Store auslagern (z. B. Redis), auf den alle Server-Instanzen zugreifen.
    - **Alternative: Sticky Sessions** – der Load Balancer sorgt dafür, dass alle Requests eines Nutzers immer an denselben Server geleitet werden.
      Nachteil: Schlechtere Lastverteilung und geringere Ausfallsicherheit (bei Server-Ausfall geht die Session verloren).



# Schichtenzuordnung

# Übung 3.5: Schichtenzuordnung

Ein Entwickler implementiert eine Bestellfunktion.

```
// (A)
<form action="/order" method="POST">
   <input name="productId" type="hidden" value="42"/>
   <button type="submit">Jetzt kaufen</button>
</form>

// (B)
if (product.getStock() < quantity)
  throw new InsufficientStockException();
double total = product.getPrice() * quantity * taxService.getRate();
emailService.sendConfirmation(user, order);

// (C)
@Query("SELECT p FROM Product p WHERE p.id = :id")
Optional<Product> findById(@Param("id") Long id);
```

Ordnen Sie die Code-Fragmente den drei Schichten des 3-Schichten-Modells zu (Präsentation / Anwendung / Persistenz) und begründen Sie Ihre Zuordnung kurz.

# Übung 3.5: Schichtenzuordnung — Lösung

| Fragment | Schicht | Begründung |
|---|---|---|
| **(A)** HTML-Formular | Präsentation | Darstellung für den Nutzer, Eingabe-Erfassung — ändert sich, wenn sich das Design ändert |
| **(B)** Lagerprüfung, Steuer, E-Mail | Anwendung | Geschäftsregeln, Validierung, Orchestrierung — ändert sich, wenn Fachlogik sich ändert |
| **(C)** JPA-Query | Persistenz | Datenbankzugriff — ändert sich, wenn die Datenbank ausgetauscht wird |

**Faustregel:**
Fragen Sie sich, welche Änderung diesen Code berühren würde:
Designänderung → Präsentation.
Neue Geschäftsregel → Anwendung.
Anderes DB-Schema → Persistenz.

# Architekturprinzipien anwenden

# Übung 3.6: Architekturprinzipien anwenden

Im folgenden Code sind die Architekturprinzipien Separation of Concerns (SoC), Single Responsibility (SRP) und Don't Repeat Yourself (DRY) verletzt:

```
@PostMapping("/checkout")
public String checkout(HttpServletRequest req, Model model) {
  String userId = req.getSession().getAttribute("userId").toString();
  ResultSet rs = db.query("SELECT * FROM cart WHERE user_id = " + userId);
  double total = 0;
  while (rs.next()) {
    total += rs.getDouble("price") * rs.getInt("qty") * 1.19;
  }
  // gleiche Steuerberechnung wie in /invoice und /quote
  String html = "<h1>Total: Euro" + total + "</h1>";
  model.addAttribute("content", html);
  sendEmail(userId, "Bestellung eingegangen, Total: Euro" + total);
  return "result";
}
```

1. Welche Prinzipien werden verletzt? Beschreiben Sie je eine konkrete Verletzung.

2. Skizzieren Sie eine verbesserte Struktur (Angabe der Klassennamen genügt).



# Übung 3.6: Architekturprinzipien anwenden — Lösung (1/2)

1. **Verletzungen:**

   - **SoC:** Der Controller enthält Datenbankzugriff (SQL), Geschäftslogik (Steuer), HTML-Erzeugung und E-Mail-Versand — vier verschiedene Verantwortlichkeiten in einer Methode.
   - **SRP:** Die Methode tut mehr als eine Sache: Sie verwaltet Session, berechnet Preise, generiert HTML und verschickt E-Mails.
   - **DRY:** Kommentar ergibt, dass `*1.19` für die Steuer auch in `/invoice` und `/quote` dupliziert ist — bei Steueränderung müssen drei Stellen geändert werden.

   *Separation of Concerns (SoC) trennt ein System in unterschiedliche Verantwortungsbereiche wie z. B. Frontend, Business-Logik und Datenhaltung (→ Systembereiche), während das Single Responsibility Principle (SRP) fordert, dass eine Klasse/Methode nur eine Aufgabe erfüllt. (→ feingranularer, z.B. auf Klassenebene)*

# Übung 3.6: Architekturprinzipien anwenden — Lösung (2/2)

2. **Verbesserte Struktur:**
   - **CheckoutController**
     → Nimmt HTTP-Request entgegen und gibt View zurück
   - **CartService**
     → Orchestriert den Checkout-Prozess (Ablauflogik)
   - **CartRepository**
     → Zugriff auf Warenkorb-Daten aus der Datenbank
   - **TaxService**
     → Berechnet Steuer (zentrale Stelle → DRY)
   - **EmailService**
     → Versand von Bestellbestätigungen
   - **HtmlView/Template (oder View-Template Engine)**
     → Zuständig für Darstellung (kein HTML im Controller)

# Git – Merge, Konfliktauflösung und .gitignore

# Übung 3.7: Git (1/3)

Verwenden Sie für diese Aufgabe das Repository aus Übung 2, Aufgabe 2, das Sie bereits lokal geklont haben. Führen Sie vor Beginn der Aufgabe einen Pull durch, um sicherzugehen, dass das lokale Repository aktuell ist.

Führen Sie alle folgenden Schritte in Ihrem lokalen Repository aus.

**1. Arbeiten mit Branches**

- Erstellen Sie im `main`-Branch eine neue Datei `notes.txt` mit Inhalt *Hallo Welt!*
- Fügen Sie die Datei zur Versionskontrolle hinzu und committen Sie diese.
- Erstellen Sie ausgehend vom aktuellen `main`-Branch einen neuen Branch `feature/update-text` und wechseln Sie auf diesen.
- Bearbeiten Sie nun die Datei `notes.txt` in beiden Branches unterschiedlich:
  - Im Branch `feature/update-text`: Ändern Sie den Inhalt zu *Version B* und committen Sie die Änderung.
  - Wechseln Sie zurück auf den Branch `main`: Ändern Sie den Inhalt zu *Version A* und committen Sie die Änderung.

# Übung 3.7: Git (1/3)

**Merge-Konflikt**

1. Führen Sie einen Merge des Branches `feature/update-text` in den `main`-Branch durch.
2. Beschreiben Sie, wie Git einen Merge-Konflikt in der Datei darstellt.
3. Lösen Sie den Merge-Konflikt so, dass die finale Datei Inhalt *Version A & Version B* hat.
4. Geben Sie die notwendigen Git-Befehle an, um die Konfliktlösung abzuschließen.

# Übung 3.7: Git (2/3)

**2. .gitignore**

In vielen Projekten sollen bestimmte Dateien nicht versioniert werden (z. B. temporäre Dateien oder Logs).

1. Erklären Sie kurz den Zweck der Datei `.gitignore`.
2. Erstellen Sie eine `.gitignore`-Datei mit folgenden Regeln:
   - Ignoriere alle Dateien mit der Endung `.log`
   - Ignoriere den Ordner `temp/`
   - Ignoriere die Datei `config.local.json`
3. Warum werden bereits getrackte Dateien nicht automatisch durch `.gitignore` ignoriert?

# Übung 3.7: Git (3/3)

### 3. Zustandsanalyse eines Repositories

Gegeben sei folgendes Szenario in einem lokalen Repository. Die Datei `.gitignore` aus der vorherigen Teilaufgabe ist bereits vorhanden. Ausgangspunkt ist ein Commit mit folgendem Inhalt der Datei `data.txt`:

```
Hello World
-
-
Hello WDE!
```

Es werden folgende Befehle ausgeführt:

```
git branch feature
git checkout feature
```

*(Fortsetzung auf der nächsten Folie)*

# Übung 3.7: Git - Zustandsanalyse (Fortsetzung)

Im Branch `feature` wird die zweite Zeile geändert und committet:

```
Hello World
-
-
Hello from Feature
```

```
git add data.txt
git commit -m "Feature change"
git checkout main
```

Im Branch `main` wird die erste Zeile geändert und committet:

```
Hello from Main
-
-
Hello WDE!
```

*(Fortsetzung auf der nächsten Folie)*

# Übung 3.7: Git - Zustandsanalyse (Fortsetzung 2)

```
git add data.txt
git commit -m "Main change"
echo "Debug info" > debug.log
git merge feature
```

1. Welchen Inhalt hat die Datei `data.txt` im `main`-Branch nach dem Merge?
2. Wird die Datei `debug.log` von Git verfolgt? Begründen Sie Ihre Antwort.
3. Wie sieht der Stand des Remote-Repositories (`origin/main`) nach diesem Ablauf aus? Begründen Sie Ihre Antwort.

# Übung 3.7: Git — Lösung (1/6)

## 1. Arbeiten mit Branches

- Datei im `main`-Branch erstellen und committen:

```
git checkout main
echo "Hallo Welt!" > notes.txt
git add notes.txt
git commit -m "Add notes.txt"
```

- Neuen Branch erstellen und wechseln:

```
git checkout -b feature/update-text
```

# Übung 3.7: Git — Lösung (2/6)

- Änderung im Feature-Branch:

```
echo "Version B" > notes.txt
git add notes.txt
git commit -m "Feature change"
```

- Zurück auf `main` wechseln und dort ändern:

```
git checkout main
echo "Version A" > notes.txt
git add notes.txt
git commit -m "Main change"
```

# Übung 3.7: Git — Lösung (3/6)

## 2. Merge-Konflikt

1. Der Konflikt entsteht beim Ausführen von:

```
git merge feature/update-text
```

2. Git stellt den Merge-Konflikt direkt in der Datei `notes.txt` dar:

```
<<<<<<< HEAD
Version A
=======
Version B
>>>>>>> feature/update-text
```

**Bedeutung der Markierungen:**

- `<<<<<<< HEAD`: Beginn des Konfliktbereichs. Der folgende Inhalt entspricht dem aktuellen Branch (`main`).
- `=======`: Trennt die beiden Versionen.
- `>>>>>>> feature/update-text`: Ende des Konfliktbereichs. Der obere Teil stammt aus dem Branch `feature/update-text`.

# Übung 3.7: Git — Lösung (4/6)

Zur Lösung des Konflikts:

- **Eigene Version behalten:** Entfernen Sie die Markierungen und den fremden Teil.
- **Andere Version übernehmen:** Entfernen Sie die Markierungen und den eigenen Teil.
- **Beide kombinieren:** Inhalt sinnvoll zusammenführen und alle Markierungen entfernen.

3. Manuelle Lösung des Konflikts:

```
Version A & Version B
```

4. Befehle zum Abschließen der Konfliktlösung:

```
git add notes.txt
git commit -m "Resolve merge conflict"
```

# Übung 3.7: Git — Lösung (5/6)

## 3. .gitignore

1. Die Datei `.gitignore` legt fest, welche Dateien oder Ordner von Git nicht verfolgt (tracked) werden sollen.

2. Beispielinhalt:

```
*.log
temp/
config.local.json
```

3. Bereits getrackte Dateien bleiben weiterhin in der Versionskontrolle, da `.gitignore` nur für ungetrackte Dateien gilt. Um sie zu ignorieren, müssen sie zuerst aus dem Index entfernt werden:

```
git rm --cached <datei>
```

# Übung 3.7: Git — Lösung (6/6)

## 4. Zustandsanalyse eines Repositories

1. Inhalt von `data.txt` nach dem Merge:

```
Hello from Main
-
-
Hello from Feature
```

   **Begründung:** Nur die jeweils committeten Änderungen beider Branches werden zusammengeführt. Da unterschiedliche Zeilen geändert wurden, führt Git den Merge automatisch aus.

2. Die Datei `debug.log` wird nicht von Git verfolgt.
   **Begründung:** Die Datei `.gitignore` enthält die Regel `*.log`. Dadurch wird die Datei von Git ignoriert. Sie wurde nicht explizit hinzugefügt.

3. Das Remote-Repository bleibt unverändert.
   **Begründung:** Es wurde kein `git push` ausgeführt. Alle Änderungen existieren nur im lokalen Repository.



# JavaScript



# Übung 3.8: JavaScript

*Hinweis:* Diese Aufgabe wird in der Übung nicht explizit behandelt. Sie dient der eigenständigen Wiederholung und Festigung der JavaScript-Kenntnisse sowie als Vorbereitung auf spätere Aufgaben und das Projekt.

Bearbeiten Sie die kostenlosen (ersten) Kapitel von learnjavascript.online. Alternativ können Sie auch den umfangreichen kostenlosen JavaScript Basiskurs von freeCodeCamp nutzen.

Eine gute Möglichkeit einzelne Aspekte von JavaScript nachzuschlagen ist w3school.com.